

模型工具

批量处理

 Copy page

❶ Batch API 专为处理大规模数据请求而设计，适用于无需即时反馈的任务。通过 Batch API，开发者可以通过文件提交大量任务，且价格降低 50%（GLM-4-Flash 免费）、无并发限制。

典型应用场景



文章分类

为大量文章、帖子或产品描述添加分类标签。



情感分析

评估客户反馈、社交媒体帖子和商品评价的情感倾向。



信息提取

从文本数据中识别和抽取关键内容。

批量使用教程

我们将通过一个情感分析的实际案例来演示如何使用 Batch API。在这个示例中，我们将使用 GLM-4 对商品评价进行情感分类（正面、中性、负面），并添加特定问题标签（如产品缺陷、配送延迟、客服态度等）。

步骤 1：创建 Batch 文件

Batch 文件的格式应为 `.jsonl`，其中每个请求占据一行（JSON 对象）。每一行包含 API 单个请求的详细信息。



```
{
  "custom_id": "request-1",
  "method": "POST",
  "url": "/v4/chat/completions",
  "body": {
    "model": "glm-4-plus",
    "messages": [
      {"role": "system", "content": "你是一个意图分类器."},
      {"role": "user", "content": "# 任务：对以下用户评论进行情感分类和特定问题标签"},
    ],
    "temperature": 0.1
  }
}
```

GLM-4V-PLUS 图像处理示例

```
{
  "custom_id": "request-1",
  "method": "POST",
  "url": "/v4/chat/completions",
  "body": {
    "model": "glm-4v-plus",
    "messages": [
      {
        "role": "system",
        "content": "You are a helpful assistant."
      },
      {
        "role": "user",
        "content": [
          {
            "type": "text",
            "text": "请描述图中的内容。"
          },
          {
            "type": "image_url",
            "image_url": {"url": "url地址或base64编码"}
          }
        ]
      }
    ],
    "max_tokens": 1000
  }
}
```

CogView-3 图像生成示例



BigModel



```
{
  "custom_id": "request-1",
  "method": "POST",
  "url": "/v4/images/generations",
  "body": {
    "model": "cogview-3",
    "prompt": "一只可爱的小猫咪"
  }
}
```

Embedding 向量化示例

```
{
  "custom_id": "request-1",
  "method": "POST",
  "url": "/v4/embeddings",
  "body": {
    "model": "embedding-2",
    "input": "你好"
  }
}
```

JSONL文件示例

构建的 .jsonl 文件如下，本示例中包含 10 个请求，单个文件最多支持 50000 个请求且大小不超过 100M：



BigModel

```
{ "custom_id": "request-1", "method": "POST", "url": "/v4/chat/completions", "body":  
{ "custom_id": "request-2", "method": "POST", "url": "/v4/chat/completions", "body":  
{ "custom_id": "request-3", "method": "POST", "url": "/v4/chat/completions", "body":  
{ "custom_id": "request-4", "method": "POST", "url": "/v4/chat/completions", "body":  
{ "custom_id": "request-5", "method": "POST", "url": "/v4/chat/completions", "body":  
{ "custom_id": "request-6", "method": "POST", "url": "/v4/chat/completions", "body":  
{ "custom_id": "request-7", "method": "POST", "url": "/v4/chat/completions", "body":  
{ "custom_id": "request-8", "method": "POST", "url": "/v4/chat/completions", "body":  
{ "custom_id": "request-9", "method": "POST", "url": "/v4/chat/completions", "body":  
{ "custom_id": "request-10", "method": "POST", "url": "/v4/chat/completions", "body":
```

步骤 2：上传 Batch 文件

首先需要将准备好的 `.jsonl` 文件上传到平台：

[python](#)

```
from zai import ZhipuAiClient  
  
client = ZhipuAiClient(api_key="your-api-key")  
  
# 上传批处理文件  
file_object = client.files.create(  
    file=open("batch_requests.jsonl", "rb"),  
    purpose="batch"  
)  
print(file_object)
```

步骤 3：创建 Batch 任务

使用上传文件的 ID 创建批处理任务：

[python](#)



```
client = ZhipuAiClient(api_key="your-api-key")
# 创建批处理任务
batch = client.batches.create(
    input_file_id=file_object.id,
    endpoint="/v4/chat/completions",
    auto_delete_input_file=True,
    metadata={
        "description": "商品评价情感分析",
        "project": "sentiment_analysis"
    }
)
print(batch)
```

步骤 4：监控任务状态

定期检查批处理任务的执行状态：

[python](#)

```
from zai import ZhipuAiClient
```

```
client = ZhipuAiClient(api_key="your-api-key")
```

```
# 检查任务状态
```

```
while True:
```

```
    batch_status = client.batches.retrieve("your_batch_id")
```

```
    print(f"任务状态: {batch_status.status}")
```

```
    if batch_status.status == "completed":
```

```
        print("任务完成! ")
```

```
        break
```

```
    elif batch_status.status in ["failed", "expired", "cancelled"]:
```

```
        print(f"任务失败, 状态: {batch_status.status}")
```

```
        break
```

```
time.sleep(30) # 等待30秒后再次检查
```

状态	描述
validating	文件正在验证中, Batch 任务未开始
failed	文件未通过验证
in_progress	文件已成功验证, Batch 任务正在进行中
finalizing	Batch 任务已完成, 结果正在准备中
completed	Batch 任务已完成, 结果已准备好
expired	Batch 任务未能完成
cancelling	Batch 任务正在取消中
cancelled	Batch 任务已取消

步骤 5: 下载结果

1. 系统会对 Batch 结果文件分开保存，请分别进行下载：

- output_file_id：保存成功执行请求的输出文件的ID。
- error_file_id：保存出现错误请求的输出文件的ID。

2. 系统只保留您的数据30天。请及时下载和备份您的数据，过期后文件将自动删除，无法恢复。

python

```
# 下载结果文件
if batch_status.status == "completed":
    result_content = client.files.content(batch_status.output_file_id)
    result_content.write_to_file("batch_results.jsonl")
    print("结果文件下载完成：batch_results.jsonl")

# 如果有错误文件，也可以下载
if batch_status.error_file_id:
    error_content = client.files.content(batch_status.error_file_id)
    error_content.write_to_file("batch_errors.jsonl")
    print("错误文件下载完成：batch_errors.jsonl")
```

最终处理完成的结果如下：



```
{ "response": { "status_code": 200, "body": { "created": 1715959701, "usage": { "completion_t": 1, "prompt_tokens": 1, "completion_tokens": 1, "total_tokens": 2 } } },  
{"response": { "status_code": 200, "body": { "created": 1715959701, "usage": { "completion_t": 1, "prompt_tokens": 1, "completion_tokens": 1, "total_tokens": 2 } } },  
{"response": { "status_code": 200, "body": { "created": 1715959701, "usage": { "completion_t": 1, "prompt_tokens": 1, "completion_tokens": 1, "total_tokens": 2 } } },  
{"response": { "status_code": 200, "body": { "created": 1715959701, "usage": { "completion_t": 1, "prompt_tokens": 1, "completion_tokens": 1, "total_tokens": 2 } } },  
{"response": { "status_code": 200, "body": { "created": 1715959701, "usage": { "completion_t": 1, "prompt_tokens": 1, "completion_tokens": 1, "total_tokens": 2 } } },  
{"response": { "status_code": 200, "body": { "created": 1715959701, "usage": { "completion_t": 1, "prompt_tokens": 1, "completion_tokens": 1, "total_tokens": 2 } } },  
{"response": { "status_code": 200, "body": { "created": 1715959701, "usage": { "completion_t": 1, "prompt_tokens": 1, "completion_tokens": 1, "total_tokens": 2 } } },  
{"response": { "status_code": 200, "body": { "created": 1715959702, "usage": { "completion_t": 1, "prompt_tokens": 1, "completion_tokens": 1, "total_tokens": 2 } } },  
{"response": { "status_code": 200, "body": { "created": 1715959701, "usage": { "completion_t": 1, "prompt_tokens": 1, "completion_tokens": 1, "total_tokens": 2 } } },  
{"response": { "status_code": 200, "body": { "created": 1715959701, "usage": { "completion_t": 1, "prompt_tokens": 1, "completion_tokens": 1, "total_tokens": 2 } } },
```

删除文件

上传 Batch 文件时，每次最多上传 1000 个。若任务量巨大，请及时删除已处理完毕的文件，以便继续上传新文件。

```
from zai import ZhipuAiClient  
  
client = ZhipuAiClient(api_key="your-api-key")  
  
result = client.files.delete(  
    file_id="文件id"  
)
```

文件限制

- 单个文件最多支持 50,000 个请求
- 文件大小不超过 100MB
- 每个 batch 文件只能包含对单个模型的请求
- 每个请求必须包含 `custom_id` 且是唯一的，用来将结果和输入进行匹配

接口信息

接口详情

- 传输方式: https
- 请求地址: https://open.bigmodel.cn/api/paas/v4/batches
- 调用方式: 同步调用
- 请求格式: JSON
- 响应格式: JSON
- 接口请求类型: POST

请求参数

参数名称	类型	是否必填	参数说明
input_file_id	string	是	上传文件的 ID，该文件包含Batch的请求输入文件必须是 .Jsonl 格式，并且文件
endpoint	string	是	Batch 中所有请求将使用的端点。目前支持 /v4/chat/completions。
completion_window	string	废弃	原有的时间参数已不再适用，新的任务预计任务将在24小时内完成，如果任务
auto_delete_input_file	bool	否	是否自动删除batch原始文件，默认为 True：执行自动删除。False：保留原
metadata	object or null	否	用于存储与 Batch 相关的数据，如客户可附加到对象上的键值对集合最多为 1

请求示例

安装最新版本

```
pip install zai-sdk
```

或指定版本

```
pip install zai-sdk==0.2.2
```

验证安装

```
import zai
print(zai.__version__)
```

```
from zai import ZhipuAiClient

client = ZhipuAiClient(api_key="") # 填写您自己的APIKey

create = client.batches.create(
    input_file_id="file_123",
    endpoint="/v4/chat/completions",
    auto_delete_input_file=True,
    metadata={
        "description": "Sentiment classification"
    }
)
print(create)
```

响应内容

返回 `Batch` 对象。

Batch 对象结构



常见问题

- ▶ Batch API的价格如何?
- ▶ Batch API 支持哪些模型和并发限制?
- ▶ 如何在调用Batch API 前进行实名认证?
- ▶ 如何下载 Batch 结果文件?
- ▶ Batch 的过期如何处理?
- ▶ Batch 文件有哪些存储限制?
- ▶ 如何删除 Batch 文件?